Programming for Artificial Intelligence

Lab 11 — Task 1

Overview of Generative AI Concepts

Rasikh Ali

Introduction

This document provides detailed explanations of eight core concepts in Generative AI and modern AI development. Each concept includes a definition, key features, real-world use cases, and a Python code example to help ground the theory in practical application.

1. LangChain

Definition

LangChain is an open-source framework designed to simplify the development of applications powered by Large Language Models (LLMs). It provides tools, abstractions, and integrations that make it easy to chain together multiple AI components — such as prompts, models, memory, and tools — into powerful, context-aware applications.

Key Features & Components

- **Chains**: Sequences of operations linking prompts, models, and utilities together to form complex workflows.
- **Agents**: Dynamic systems that decide which tools or actions to use based on user input, enabling autonomous behavior.
- **Memory**: Enables LLMs to remember previous interactions, making chatbots and assistants context-aware across conversations.
- **Document Loaders & Retrievers**: Load data from PDFs, websites, or databases and retrieve relevant chunks for answering questions.
- **Tool Integration**: Easily integrates with APIs, databases, vector stores (like FAISS, ChromaDB), and web search tools.

Common Use Cases

Building intelligent chatbots, question-answering systems over documents, automated research assistants, and AI agents that interact with external services.

Code Example (Python)

```
from langchain.llms import OpenAI
from langchain.chains import LLMChain
from langchain.prompts import PromptTemplate


llm = OpenAI(temperature=0.7)
```

```
prompt = PromptTemplate(input_variables=["topic"], template="Explain {topic} in simple
terms.")
chain = LLMChain(llm=llm, prompt=prompt)
print(chain.run("machine learning"))
```

---

2. LLMs (Large Language Models)

Definition

Large Language Models (LLMs) are deep learning models trained on massive corpora of text data — often billions or trillions of words — to understand and generate human-like language. They are built primarily on the Transformer architecture and learn statistical patterns of language at an enormous scale.

Key Features & Components

- **Pre-training**: Trained on huge datasets (books, websites, code) using self-supervised learning to predict the next token or fill in masked tokens.
- **Fine-tuning**: After pre-training, LLMs can be fine-tuned on specific datasets for specialized tasks (e.g., medical QA, legal summarization).
- **In-Context Learning**: LLMs can perform new tasks with just a few examples provided in the prompt, without retraining.
- **Emergent Abilities**: At large scales, LLMs develop abilities not explicitly trained for, such as reasoning, analogy-making, and code generation.
- **Examples**: GPT-4 (OpenAI), PaLM (Google), LLaMA (Meta), Claude (Anthropic), Falcon, Mistral.

Common Use Cases

Text generation and completion, chatbots and virtual assistants, summarization, translation, sentiment analysis, code generation, and question answering.

Code Example (Python)

```
from transformers import pipeline


# Load a text generation pipeline using GPT-2
generator = pipeline("text-generation", model="gpt2")
result = generator("Artificial intelligence is", max_length=50)
print(result[0]["generated_text"])
```

---

3. RAG (Retrieval-Augmented Generation)

Definition

Retrieval-Augmented Generation (RAG) is an AI architecture that enhances LLMs by combining them with a retrieval mechanism. Instead of relying solely on knowledge baked into the model during training, RAG systems dynamically fetch relevant information from external sources (documents, databases, PDFs) at query time and feed it to the LLM as context before generating a response.

Key Features & Components

- **Retriever Component**: Searches a vector database or document store to find the most relevant chunks of information based on the user's query.
- **Generator Component**: An LLM that receives both the user query and the retrieved context to generate an accurate, grounded response.
- **Up-to-Date Information**: Unlike standard LLMs frozen at a training cutoff, RAG can work with real-time or frequently updated documents.
- **Reduced Hallucination**: Grounding answers in retrieved facts reduces the LLM's tendency to make things up.
- **Scalability**: Can work across thousands of documents without needing to retrain the model.

Common Use Cases

Enterprise chatbots that answer questions from internal documents, legal and medical QA systems, customer support bots, and academic research assistants.

Code Example (Python)

```
# Pseudocode for RAG pipeline
# Step 1: Convert documents to embeddings and store in VectorDB
vectordb.add_documents(documents)

# Step 2: User asks a question
query = "What is the refund policy?"

# Step 3: Retrieve relevant chunks
relevant_chunks = vectordb.similarity_search(query, k=3)

# Step 4: Pass retrieved context + query to LLM
response = llm.generate(context=relevant_chunks, question=query)
print(response)
```

---

4. FAISS (Facebook AI Similarity Search)

Definition

FAISS (Facebook AI Similarity Search) is an open-source library developed by Meta AI for performing fast and efficient similarity search over large collections of high-dimensional vectors (embeddings). It is a core component in many AI pipelines that require searching through millions or billions of embeddings quickly.

Key Features & Components

- **Exact and Approximate Search**: Supports both exact nearest-neighbor search (precise but slow) and approximate nearest-neighbor (ANN) search (fast with slight accuracy trade-off).
- **Indexing Methods**: Provides various index types such as Flat (brute-force), IVF (inverted file), HNSW (hierarchical navigable small world), and PQ (product quantization) for speed/accuracy trade-offs.
- **GPU Support**: FAISS can run on GPUs for massive speedups when dealing with billions of vectors.
- **Integration**: Works seamlessly with LangChain, Hugging Face, and custom Python pipelines.

- **Scalability**: Designed to handle datasets too large to fit in memory using on-disk indexes.

Common Use Cases

Document retrieval in RAG systems, duplicate detection, image similarity search, recommendation systems, and semantic search engines.

Code Example (Python)

```
import faiss
import numpy as np

# Create 10,000 random 128-dimensional vectors
d = 128  # vector dimension
nb = 10000  # number of vectors
nq = 5   # number of query vectors

np.random.seed(42)
xb = np.random.random((nb, d)).astype("float32")
xq = np.random.random((nq, d)).astype("float32")

# Build FAISS index
index = faiss.IndexFlatL2(d)  # L2 distance metric
index.add(xb)  # add vectors to index

# Search for 3 nearest neighbors
k = 3
D, I = index.search(xq, k)
print("Nearest neighbor indices:", I)
print("Distances:", D)
```

---

5. Vector (in AI Context)

Definition

In Artificial Intelligence, a vector is a fixed-length list (array) of floating-point numbers that encodes the semantic meaning of data — such as words, sentences, images, or documents — in a multi-dimensional mathematical space. These numeric representations are often called embeddings. The fundamental insight is that semantically similar items will have vectors that are close to each other in this high-dimensional space.

Key Features & Components

- **Semantic Encoding**: Words or sentences with similar meanings will have vectors pointing in similar directions (high cosine similarity).
- **Dimensionality**: Embedding vectors typically range from 128 to 1536 dimensions depending on the model used.
- **Created by Embedding Models**: Models like BERT, Word2Vec, Sentence-BERT, OpenAI Ada, and CLIP convert raw data into vectors.
- **Distance Metrics**: Similarity between vectors is measured using Cosine Similarity, Euclidean Distance (L2), or Dot Product.

- **Universality**: Vectors can represent text, images, audio, video, or any type of data in a unified numeric format.

Common Use Cases

Semantic search, document clustering, recommendation engines, similarity detection, cross-modal search (text-to-image), and as the foundation for RAG and VectorDB systems.

Code Example (Python)

```
from sentence_transformers import SentenceTransformer
import numpy as np

model = SentenceTransformer("all-MiniLM-L6-v2")

# Convert sentences to vectors
sentences = ["I love machine learning", "AI is transforming the world", "I enjoy pizza"]
vectors = model.encode(sentences)

print(f"Vector shape: {vectors.shape}")  # (3, 384)

# Compute cosine similarity between sentence 0 and sentence 1
from sklearn.metrics.pairwise import cosine_similarity
sim = cosine_similarity([vectors[0]], [vectors[1]])
print(f"Similarity score: {sim[0][0]:.4f}")
```

---

6. VectorDB (Vector Database)

Definition

A Vector Database is a specialized database system designed to store, index, and efficiently query high-dimensional vector embeddings. Unlike traditional relational databases that search by exact keyword match, vector databases search by semantic similarity — finding vectors (and their associated content) that are most similar to a query vector. They are a critical infrastructure component for modern AI applications.

Key Features & Components

- **Similarity Search**: Performs Approximate Nearest Neighbor (ANN) search to find the most semantically relevant entries.
- **Persistent Storage**: Unlike in-memory FAISS indexes, vector databases persist data to disk and support CRUD operations.
- **Metadata Filtering**: Allows combining vector similarity with traditional filters (e.g., find similar documents where category='finance').
- **Scalability**: Designed to scale to billions of vectors across distributed systems.
- **Popular Options**: Pinecone (managed cloud), Weaviate (open-source, hybrid search), ChromaDB (lightweight, local), Qdrant (Rust-based, fast), Milvus (enterprise-grade).

Common Use Cases

Semantic search engines, RAG pipelines, recommendation systems, fraud detection, personalization engines, and multi-modal AI applications.

Code Example (Python)

```
import chromadb
from chromadb.utils import embedding_functions

# Initialize ChromaDB
client = chromadb.Client()

# Create a collection
collection = client.create_collection("my_documents")

# Add documents (ChromaDB auto-generates embeddings)
collection.add(
    documents=["Python is a programming language", "Machine learning uses data"],
    ids=["doc1", "doc2"]
)

# Query for similar documents
results = collection.query(
    query_texts=["What is Python?"],
    n_results=2
)
print(results)
```

---

7. Generative AI (GenAI)

Definition

Generative Artificial Intelligence (GenAI) refers to a class of AI systems capable of creating new, original content — including text, images, audio, video, code, and 3D models — by learning patterns and distributions from existing training data. Unlike discriminative AI (which classifies or predicts), generative AI produces novel outputs that did not previously exist.

Key Features & Components

- **Content Creation**: Can generate human-quality text, photorealistic images, music compositions, and functional code.
- **Foundation Models**: Built on large pre-trained models (LLMs, diffusion models) that can be adapted for many tasks via prompting or fine-tuning.
- **Multimodal Capabilities**: Modern GenAI systems like GPT-4V and Gemini can process and generate across multiple modalities (text + images + audio).
- **Prompt Engineering**: The quality and specificity of the input prompt heavily influences the output quality.
- **Categories**: Text GenAI (ChatGPT, Claude), Image GenAI (DALL-E 3, Stable Diffusion, Midjourney), Code GenAI (GitHub Copilot, Codex), Audio GenAI (MusicLM, Suno).

Common Use Cases

Content creation and copywriting, code generation and debugging, image and art generation, synthetic data generation for training AI models, drug discovery, game design, and personalized education.

Code Example (Python)

```
from transformers import pipeline

# Text Generation using Hugging Face pipeline
generator = pipeline("text-generation", model="gpt2", max_new_tokens=100)

prompt = "The future of artificial intelligence in healthcare is"
result = generator(prompt, num_return_sequences=1)

print("Generated Text:")
print(result[0]["generated_text"])
```

---

8. GANs (Generative Adversarial Networks)

Definition

Generative Adversarial Networks (GANs) are a class of deep learning architectures introduced by Ian Goodfellow et al. in 2014. They consist of two neural networks — a Generator and a Discriminator — that are trained simultaneously in a competitive (adversarial) setting. This competition drives both networks to improve until the generator produces outputs indistinguishable from real data.

Key Features & Components

- **Generator Network**: Takes random noise (latent vector z) as input and learns to generate realistic data (images, audio, etc.) that fools the Discriminator.
- **Discriminator Network**: Acts as a binary classifier that learns to distinguish between real data (from the training set) and fake data (from the Generator).
- **Adversarial Training**: The two networks compete — the Generator tries to fool the Discriminator, while the Discriminator tries not to be fooled. This adversarial dynamic improves both.
- **Latent Space**: The Generator maps a continuous latent space of random vectors to the data space, enabling interpolation and controlled generation.
- **GAN Variants**: DCGAN (deep convolutional GAN), StyleGAN (high-quality face synthesis), CycleGAN (image-to-image translation), Pix2Pix, BigGAN, and Conditional GANs (cGAN).

Common Use Cases

Photorealistic image generation, face synthesis (deepfakes), image-to-image translation, super-resolution, data augmentation, art generation, medical image synthesis, and video generation.

Code Example (Python)

```
import torch
import torch.nn as nn

# Simple GAN architecture

# Generator: noise -> fake image
class Generator(nn.Module):
    def __init__(self, noise_dim=100, output_dim=784):
```

```
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(noise_dim, 256),
            nn.ReLU(),
            nn.Linear(256, output_dim),
            nn.Tanh()  # output in [-1, 1]
        )
    def forward(self, z):
        return self.net(z)

# Discriminator: image -> real/fake probability
class Discriminator(nn.Module):
    def __init__(self, input_dim=784):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, 256),
            nn.LeakyReLU(0.2),
            nn.Linear(256, 1),
            nn.Sigmoid()  # output probability
        )
    def forward(self, x):
        return self.net(x)

G = Generator()
D = Discriminator()
print("Generator:", G)
print("Discriminator:", D)
```